INTEGRATING AZURE SERVICES WITH A BLAZOR APP

Instructor : Muhammad Sudais

Table of Contents

Blazor App File Upload to Azure Blob Storage

1. Set Up Azure Blob Storage

1. **Create a Blob Storage Account**:
    - Go to the Azure Portal and create a new Blob Storage account.
    - Note down the connection string from the Azure Portal. You will need it to configure your Blazor app.
2. **Create a Blob Container**:
    - Within your Blob Storage account, create a container where you'll store your files.

2. Create a Blazor WebAssembly App

1. **Create a New Blazor WebAssembly App**:
    - Open Visual Studio and create a new Blazor WebAssembly App project.
2. **Add Required NuGet Packages**:
    - Add the Azure.Storage.Blobs NuGet package to your Blazor project. This package allows you to interact with Azure Blob Storage.

dotnet add package Azure.Storage.Blobs

3. Configure Blob Storage in Blazor

1. **Add Azure Storage Configuration**:

In appsettings.json, add your Azure Blob Storage connection string and container name:

```
{
 "AzureBlobStorage": {
  "ConnectionString": "YourAzureBlobStorageConnectionString",
  "ContainerName": "your-container-name"
 }
}
```

2. **Create a Service to Handle Blob Storage**:

Create a service class to interact with Azure Blob Storage. Add a new class BlobService.cs:

```
using Azure.Storage.Blobs;
using System.IO;
using System.Threading.Tasks;

public class BlobService
{
  private readonly string _connectionString;
  private readonly string _containerName;

  public BlobService(string connectionString, string containerName)
  {
    _connectionString = connectionString;
    _containerName = containerName;
  }

  public async Task UploadFileAsync(Stream fileStream, string fileName)
```

```
  {
    var blobServiceClient = new BlobServiceClient(_connectionString);
    var blobContainerClient = blobServiceClient.GetBlobContainerClient(_containerName);
    var blobClient = blobContainerClient.GetBlobClient(fileName);

    await blobClient.UploadAsync(fileStream, true);
  }
}
```

Register this service in Program.cs:

```
builder.Services.AddSingleton(new BlobService(
  builder.Configuration["AzureBlobStorage:ConnectionString"],
  builder.Configuration["AzureBlobStorage:ContainerName"]
));
```

4. Create a File Upload Component

1. **Add a File Upload Component**:

Create a new Razor component FileUpload.razor:

```
@page "/fileupload"
@inject BlobService BlobService

<h3>File Upload</h3>

<input type="file" @onchange="HandleFileChange" />
<button @onclick="UploadFile">Upload</button>

@code {
  private IBrowserFile file;

  private void HandleFileChange(InputFileChangeEventArgs e)
  {
    file = e.File;
  }

  private async Task UploadFile()
  {
    if (file != null)
    {
      using (var stream = file.OpenReadStream())
      {
        await BlobService.UploadFileAsync(stream, file.Name);
      }
    }
  }
}
```

5. Test Your Blazor Application

1. **Run Your Application**:
    - Start your Blazor WebAssembly app in Visual Studio.
2. **Navigate to the File Upload Page**:
    - Go to /fileupload in your browser.
3. **Upload a File**:
    - Use the file input to select a file and click the upload button.

4. **Verify File Upload**:
    - Check your Azure Blob Storage container to ensure the file has been uploaded.

Create a Function App that moves this file from Blob to File Storage.

To create an Azure Function App that moves a file from Azure Blob Storage to Azure File Storage, follow these steps:

1. Set Up Azure File Storage

1. **Create a File Storage Account**:
    - Go to the Azure Portal and create a new Azure File Storage account.
    - Note down the connection string for Azure File Storage.
2. **Create a File Share**:
    - Within your Azure File Storage account, create a file share where you'll move the files.

2. Create an Azure Function App

1. **Create a New Function App**:
    - Open Visual Studio or the Azure Portal and create a new Azure Function App project.
    - Choose a trigger type, such as a Timer trigger or Blob trigger, depending on how you want to invoke the function.
2. **Add Required NuGet Packages**:
    - Add the following NuGet packages to your Function App project to interact with Azure Blob Storage and Azure File Storage:
        - Azure.Storage.Blobs
        - Azure.Storage.Files.Shares

You can add them via the NuGet Package Manager in Visual Studio or by running:

dotnet add package Azure.Storage.Blobs

dotnet add package Azure.Storage.Files.Shares

3. Implement the Azure Function

Here's an example of a Timer-triggered Azure Function that moves files from Azure Blob Storage to Azure File Storage:

1. **Create the Function**:
    - Add a new function to your project. For this example, let's use a Timer trigger.
2. **Function Code**:

```
using Azure.Storage.Blobs;
using Azure.Storage.Files.Shares;
using Microsoft.Azure.WebJobs;
using Microsoft.Extensions.Logging;
using System;
using System.IO;
```

```
using System.Threading.Tasks;

public static class MoveBlobToFileStorage
{
  private static readonly string blobConnectionString =
Environment.GetEnvironmentVariable("BlobStorageConnectionString");
  private static readonly string fileShareConnectionString =
Environment.GetEnvironmentVariable("FileStorageConnectionString");
  private static readonly string blobContainerName = "source-container";
  private static readonly string fileShareName = "destination-share";

  [FunctionName("MoveBlobToFileStorage")]
  public static async Task Run([TimerTrigger("0 */5 * * * *")] TimerInfo myTimer, ILogger log)
  {
    log.LogInformation($"C# Timer trigger function executed at: {DateTime.Now}");

    // Set up Blob and File clients
    var blobServiceClient = new BlobServiceClient(blobConnectionString);
    var blobContainerClient = blobServiceClient.GetBlobContainerClient(blobContainerName);

    var shareServiceClient = new ShareServiceClient(fileShareConnectionString);
    var shareClient = shareServiceClient.GetShareClient(fileShareName);

    await foreach (var blobItem in blobContainerClient.GetBlobsAsync())
    {
      var blobClient = blobContainerClient.GetBlobClient(blobItem.Name);
      var fileClient = shareClient.GetDirectoryClient("").GetFileClient(blobItem.Name);

      using (var stream = new MemoryStream())
      {
        // Download blob to memory stream
        await blobClient.DownloadToAsync(stream);
        stream.Position = 0;

        // Upload to Azure File Storage
        await fileClient.CreateAsync(stream.Length);
        await fileClient.UploadAsync(stream, overwrite: true);

        // Optionally, delete the blob after moving
        await blobClient.DeleteIfExistsAsync();
      }

      log.LogInformation($"Moved blob {blobItem.Name} to file share.");
    }
  }
}
```

3. **Configure Settings**:
    - **Add Application Settings**:
        - Go to the Azure Portal and configure your Function App's Application Settings to include:
            - BlobStorageConnectionString: The connection string for your Azure Blob Storage account.
            - FileStorageConnectionString: The connection string for your Azure File Storage account.

4. Deploy and Test Your Function App

1. **Deploy Your Function App**:
    - Deploy the Function App using Visual Studio or the Azure CLI.
2. **Test the Function**:
    - Check the Function App's logs to ensure it's running as expected.
    - Verify that files are being moved from Azure Blob Storage to Azure File Storage.

Integrate Keyvault

To integrate Azure Key Vault with the web app and function app, you will need to modify the code to use secrets from the Key Vault. Here's how you can achieve that:

1. Web App: Uploading File to Azure Blob Storage

Add Key Vault Integration:

1. **Update appsettings.json with Key Vault Endpoint:**

```
{
 "KeyVault": {
  "Endpoint": "https://<YourKeyVaultName>.vault.azure.net/"
 },
 "BlobStorage": {
  "ConnectionString": "<YourBlobStorageConnectionString>",
  "ContainerName": "<YourBlobContainerName>"
 }
}
```

2. **Configure Key Vault Access in Program.cs:**

```
using Azure.Identity;
using Azure.Security.KeyVault.Secrets;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using System;

public class Program
{
  public static void Main(string[] args)
  {
    CreateHostBuilder(args).Build().Run();
  }

  public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
      .ConfigureAppConfiguration((context, config) =>
      {
        var builtConfig = config.Build();
        var keyVaultEndpoint = builtConfig["KeyVault:Endpoint"];

        config.AddAzureKeyVault(new Uri(keyVaultEndpoint), new DefaultAzureCredential());
      })
      .ConfigureWebHostDefaults(webBuilder =>
      {
        webBuilder.UseStartup<Startup>();
      });
}
```

3. **Access Secrets in the Web App to Upload Files:**

In Startup.cs, add services for blob storage:

```
public class Startup
{
```

```
  public void ConfigureServices(IServiceCollection services)
  {
    services.AddControllersWithViews();
    services.AddSingleton<BlobServiceClient>(provider =>
    {
      var configuration = provider.GetRequiredService<IConfiguration>();
      var connectionString = configuration["BlobStorage:ConnectionString"];
      return new BlobServiceClient(connectionString);
    });
  }
}
```

In the controller where you upload files:

```
using Azure.Storage.Blobs;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Configuration;
using System.IO;
using System.Threading.Tasks;

public class UploadController : Controller
{
  private readonly BlobServiceClient _blobServiceClient;
  private readonly string _containerName;

  public UploadController(BlobServiceClient blobServiceClient, IConfiguration configuration)
  {
    _blobServiceClient = blobServiceClient;
    _containerName = configuration["BlobStorage:ContainerName"];
  }

  [HttpPost("upload")]
  public async Task<IActionResult> UploadFile(IFormFile file)
  {
    if (file == null || file.Length == 0)
      return BadRequest("No file uploaded.");

    var blobContainerClient = _blobServiceClient.GetBlobContainerClient(_containerName);
    var blobClient = blobContainerClient.GetBlobClient(file.FileName);

    using (var stream = file.OpenReadStream())
    {
      await blobClient.UploadAsync(stream);
    }

    return Ok("File uploaded successfully.");
  }
}
```

2. Function App: Moving File from Blob Storage to File Storage

Add Key Vault Integration:

1. **Update local.settings.json for Local Development:**

```
{
 "IsEncrypted": false,
 "Values": {
  "AzureWebJobsStorage": "<YourStorageConnectionString>",
  "KeyVaultEndpoint": "https://<YourKeyVaultName>.vault.azure.net/",
```

```
  "BlobStorageConnectionString": "<YourBlobStorageConnectionString>",
  "FileStorageConnectionString": "<YourFileStorageConnectionString>",
  "BlobContainerName": "<YourBlobContainerName>",
  "FileShareName": "<YourFileShareName>"
 }
}
```

2. **Configure Key Vault Access in the Function App:**

```
using Azure.Identity;
using Azure.Security.KeyVault.Secrets;
using Microsoft.Azure.WebJobs;
using Microsoft.Extensions.Logging;
using System;
using System.IO;
using System.Threading.Tasks;

public static class MoveBlobToFileStorage
{
  [FunctionName("MoveBlobToFileStorage ")]
  public static async Task Run([TimerTrigger("0 */5 * * * *")] TimerInfo myTimer, ILogger log)
  {
    var keyVaultEndpoint = Environment.GetEnvironmentVariable("KeyVaultEndpoint");
    var client = new SecretClient(new Uri(keyVaultEndpoint), new DefaultAzureCredential());

    var blobStorageConnectionString = (await client.GetSecretAsync("BlobStorageConnectionString")).Value.Value;
    var fileStorageConnectionString = (await client.GetSecretAsync("FileStorageConnectionString")).Value.Value;
    var blobContainerName = (await client.GetSecretAsync("BlobContainerName")).Value.Value;
    var fileShareName = (await client.GetSecretAsync("FileShareName")).Value.Value;

    var blobServiceClient = new BlobServiceClient(blobStorageConnectionString);
    var fileShareClient = new ShareClient(fileStorageConnectionString, fileShareName);

    var blobContainerClient = blobServiceClient.GetBlobContainerClient(blobContainerName);
    await foreach (var blobItem in blobContainerClient.GetBlobsAsync())
    {
      var blobClient = blobContainerClient.GetBlobClient(blobItem.Name);
      var fileClient = fileShareClient.GetRootDirectoryClient().GetFileClient(blobItem.Name);

      using (var stream = new MemoryStream())
      {
        await blobClient.DownloadToAsync(stream);
        stream.Position = 0;
        await fileClient.CreateAsync(stream.Length);
        await fileClient.UploadAsync(stream);
      }

      log.LogInformation($"Moved blob '{blobItem.Name}' to file storage.");
    }
  }
}
```

Integrate Cosmos DB

1. Store file in Blob Storage.
2. Store metadata (filename, filesize, upload time, username) in Cosmos DB.
3. Retrieve Azure Storage connection strings and Cosmos DB credentials from Key Vault.

Web App Code (Blazor Component for Upload)

```
@page "/upload"

@inject NavigationManager Navigation
@inject IConfiguration Configuration
@inject IAzureBlobService BlobService
@inject ICosmosDbService CosmosDbService

<h3>Upload File</h3>

<RadzenTemplateForm TItem="UploadModel" Data="@uploadModel" Submit="@OnSubmit">
  <RadzenFieldset>
    <RadzenFileUpload Multiple="false" Accept="image/*"
              Change="@OnFileChange" Style="margin-bottom: 20px;" />
    <RadzenTextBox @bind-Value="uploadModel.UserName" Placeholder="Enter your username" />
    <RadzenButton Text="Upload" ButtonType="ButtonType.Submit" />
  </RadzenFieldset>
</RadzenTemplateForm>

@code {
  private UploadModel uploadModel = new UploadModel();

  private async Task OnFileChange(RadzenFileUploadChangeEventArgs e)
  {
    var file = e.Files.FirstOrDefault();
    if (file != null)
    {
      uploadModel.File = file;
    }
  }

  private async Task OnSubmit()
  {
    if (uploadModel.File != null)
    {
      // Upload file to Azure Blob Storage
      var blobUri = await BlobService.UploadFileToBlobAsync(uploadModel.File);

      // Get file metadata
      var fileSize = uploadModel.File.Size;
      var uploadTime = DateTime.UtcNow;

      // Save metadata to Cosmos DB
      var fileMetadata = new FileMetadata
      {
        FileName = uploadModel.File.Name,
        FileSize = fileSize,
```

```
            UploadTime = uploadTime,
            UserName = uploadModel.UserName
          };
          await CosmosDbService.InsertMetadataAsync(fileMetadata);
      }
    }
    public class UploadModel
    {
      public RadzenFile File { get; set; }
      public string UserName { get; set; }
    }
}
```

Blob Service Code (For File Upload)

```
public class AzureBlobService : IAzureBlobService
{
    private readonly BlobServiceClient _blobServiceClient;
    private readonly string _containerName = "uploads";

    public AzureBlobService(BlobServiceClient blobServiceClient)
    {
        _blobServiceClient = blobServiceClient;
    }

    public async Task<Uri> UploadFileToBlobAsync(RadzenFile file)
    {
        var blobContainerClient = _blobServiceClient.GetBlobContainerClient(_containerName);
        var blobClient = blobContainerClient.GetBlobClient(file.Name);

        using var stream = file.OpenReadStream();
        await blobClient.UploadAsync(stream);
        return blobClient.Uri;
    }
}
```

Cosmos DB Service Code (For Storing Metadata)

```
public class CosmosDbService : ICosmosDbService
{
    private readonly Container _container;

    public CosmosDbService(CosmosClient cosmosClient, string databaseId, string containerId)
    {
        _container = cosmosClient.GetContainer(databaseId, containerId);
    }

    public async Task InsertMetadataAsync(FileMetadata metadata)
    {
        await _container.CreateItemAsync(metadata, new PartitionKey(metadata.UserName));
    }
}

public class FileMetadata
{
    public string FileName { get; set; }
    public long FileSize { get; set; }
    public DateTime UploadTime { get; set; }
    public string UserName { get; set; }
}
```

Azure Function App (Moving Blob to File Storage)

Integrating Azure Services with a Blazor App

You can modify the Azure Function App to fetch Key Vault secrets and use them in storage operations:

```
public class MoveBlobToFileStorage
{
  private readonly string _blobConnectionString;
  private readonly string _fileStorageConnectionString;

  public MoveBlobToFileStorage(IKeyVaultService keyVaultService)
  {
    _blobConnectionString = keyVaultService.GetSecret("BlobStorageConnectionString");
    _fileStorageConnectionString = keyVaultService.GetSecret("FileStorageConnectionString");
  }

  [FunctionName("MoveBlobToFileStorage ")]
  public async Task Run([BlobTrigger("uploads/{name}")] Stream blobStream, string name, ILogger log)
  {
    // Access Blob Storage and File Storage using secrets from Key Vault
    var fileStorageClient = new FileStorageClient(_fileStorageConnectionString);
    var blobClient = new BlobClient(_blobConnectionString, "uploads", name);

    using var memoryStream = new MemoryStream();
    await blobClient.DownloadToAsync(memoryStream);
    memoryStream.Position = 0;

    await fileStorageClient.UploadFileAsync(name, memoryStream);
    log.LogInformation($"Blob {name} moved to file storage.");
  }
}
```

Azure Key Vault Integration

Both services (Web App and Function App) will use Azure Key Vault to retrieve connection strings:

```
public class KeyVaultService : IKeyVaultService
{
  private readonly SecretClient _secretClient;

  public KeyVaultService(SecretClient secretClient)
  {
    _secretClient = secretClient;
  }

  public string GetSecret(string secretName)
  {
    var secret = _secretClient.GetSecret(secretName);
    return secret.Value.Value;
  }
}
```

Configuration for Key Vault in Startup.cs

```
services.AddSingleton<IKeyVaultService, KeyVaultService>(provider =>
{
  var secretClient = new SecretClient(new Uri("<YourKeyVaultUrl>"), new DefaultAzureCredential());
  return new KeyVaultService(secretClient);
});
```

Integrate Application Insights

To add Application Insights telemetry to both your Radzen web app and Azure Function app, follow these steps:

1. Add Application Insights to Radzen Web App

Prerequisites:

- Install the necessary Application Insights NuGet package.

Install-Package Microsoft.ApplicationInsights.AspNetCore

Steps:

1. **Configure Application Insights in the Startup file:**

In your Startup.cs, configure Application Insights by adding the following lines in ConfigureServices:

```
public void ConfigureServices(IServiceCollection services)
{
  // Add Application Insights
  services.AddApplicationInsightsTelemetry(Configuration["ApplicationInsights:InstrumentationKey"]);

  // Other services
  services.AddRazorPages();
  services.AddServerSideBlazor();
}
```

Ensure that the appsettings.json file contains the Application Insights Instrumentation Key:

```
{
  "ApplicationInsights": {
    "InstrumentationKey": "<your-instrumentation-key>"
  }
}
```

2. **Track Events and Logs:**

To log telemetry events in your web app, inject TelemetryClient into your components or services:

```
using Microsoft.ApplicationInsights;

public class FileUploadService
{
  private readonly TelemetryClient _telemetryClient;

  public FileUploadService(TelemetryClient telemetryClient)
  {
    _telemetryClient = telemetryClient;
  }

  public async Task UploadFileAsync(BlobContainerClient blobContainerClient, string filePath)
  {
    try
    {
      // File upload logic
      _telemetryClient.TrackEvent("FileUploaded", new Dictionary<string, string>
      {
```

```
            { "FileName", filePath }
        });
    }
    catch (Exception ex)
    {
        _telemetryClient.TrackException(ex);
        throw;
    }
  }
}
```

3. **View Telemetry in Azure:**

Once the web app is running, telemetry will start flowing to Application Insights, and you can view logs, events, and traces in the Azure portal.

2. Add Application Insights to Azure Function App

Prerequisites:

- Install the Application Insights SDK for Azure Functions.

Install-Package Microsoft.Azure.WebJobs.Logging.ApplicationInsights

Steps:

1. **Configure Application Insights in host.json:**

Add Application Insights configuration in your host.json file:

```
{
 "logging": {
  "applicationInsights": {
   "samplingSettings": {
    "isEnabled": true
   },
   "instrumentationKey": "<your-instrumentation-key>"
  }
 }
}
```

2. **Log Custom Events and Metrics:**

To manually track telemetry data, inject the TelemetryClient into your function:

```
using Microsoft.ApplicationInsights;
using Microsoft.Azure.WebJobs;
using Microsoft.Extensions.Logging;

public class MoveBlobToFileStorage
{
  private readonly TelemetryClient _telemetryClient;

  public MoveBlobToFileStorage (TelemetryClient telemetryClient)
  {
    _telemetryClient = telemetryClient;
  }

  [FunctionName("MoveBlobToFileStorage ")]
  public async Task Run([BlobTrigger("input-container/{name}")] Stream inputBlob, string name, ILogger log)
```

```
  {
    try
    {
      // Move file logic

      _telemetryClient.TrackEvent("FileMoved", new Dictionary<string, string>
      {
        { "FileName", name }
      });
    }
    catch (Exception ex)
    {
      _telemetryClient.TrackException(ex);
      throw;
    }
  }
}
```

Now, both your Radzen web app and Function app will be connected to Application Insights, allowing you to track metrics, events, and exceptions in the Azure portal.

Connect Login with Microsoft Account

To enable Microsoft Account (Azure Active Directory - AD) authentication in your Radzen Blazor web app and retrieve the user's name, follow these steps:

1. Set Up Azure AD Authentication

Step 1: Register the Application in Azure Active Directory

1. Go to the Azure Portal and navigate to **Azure Active Directory**.
2. Under **Manage**, select **App registrations**.
3. Click **New registration**.
4. Fill in the following fields:
    - **Name**: Give your application a name.
    - **Supported account types**: Choose which accounts can use the application. Select **Accounts in this organizational directory only** if it's for your organization.
    - **Redirect URI**: For a Blazor WebAssembly app, use https://localhost:5001/authentication/login-callback.
5. Click **Register**.

Step 2: Configure API Permissions

1. Go to your app registration and select **API Permissions**.
2. Click **Add a permission**, then choose **Microsoft Graph**.
3. Add **User.Read** permission under **Delegated permissions**.

Step 3: Configure Client Secrets

1. Go to **Certificates & secrets**.
2. Click **New client secret**, and copy the value of the secret once it is created. This will be used later in your app.

Step 4: Configure Authentication

1. In the **Authentication** section of your registered app, under **Platform configurations**, click **Add a platform**.
2. Choose **Single-page application (SPA)** and add a redirect URI like https://localhost:5001/authentication/login-callback.

2. Configure Radzen Blazor App for Azure AD Authentication

Step 1: Install NuGet Packages

Add the necessary NuGet packages for Azure AD authentication:

Install-Package Microsoft.AspNetCore.Authentication.AzureAD.UI

Install-Package Microsoft.Identity.Web

Install-Package Microsoft.AspNetCore.Authentication.OpenIdConnect

Step 2: Configure Authentication in Startup.cs

Modify the Startup.cs to include Azure AD authentication:

```
using Microsoft.AspNetCore.Authentication;
using Microsoft.Identity.Web;

public void ConfigureServices(IServiceCollection services)
{
  services.AddAuthentication(AzureADDefaults.AuthenticationScheme)
      .AddAzureAD(options => Configuration.Bind("AzureAd", options));

  services.AddRazorPages();
  services.AddServerSideBlazor().AddMicrosoftIdentityConsentHandler();
  services.AddControllersWithViews();
  services.AddAuthorization();
}

public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
  app.UseAuthentication();
  app.UseAuthorization();

  app.UseEndpoints(endpoints =>
  {
    endpoints.MapControllers();
    endpoints.MapBlazorHub();
    endpoints.MapFallbackToPage("/_Host");
  });
}
```

Step 3: Add Configuration for Azure AD in appsettings.json

Add Azure AD configuration to your appsettings.json:

```
{
 "AzureAd": {
  "Instance": "https://login.microsoftonline.com/",
  "Domain": "<your-tenant-domain>",
  "TenantId": "<your-tenant-id>",
  "ClientId": "<your-client-id>",
  "CallbackPath": "/signin-oidc",
  "ClientSecret": "<your-client-secret>"
 }
}
```

3. Retrieve User Information

You can retrieve the signed-in user's name from Azure AD in your components by injecting the AuthenticationStateProvider and UserManager classes.

Here's an example of how you can retrieve the user's name and display it on your page:

```
@page "/upload"
```

```
@using Microsoft.AspNetCore.Components.Authorization
@inject AuthenticationStateProvider AuthenticationStateProvider
@inject Microsoft.AspNetCore.Identity.UserManager<IdentityUser> UserManager

<h3>Upload File</h3>

<p>User: @userName</p>

@code {
  private string userName;

  protected override async Task OnInitializedAsync()
  {
    var authState = await AuthenticationStateProvider.GetAuthenticationStateAsync();
    var user = authState.User;

    if (user.Identity.IsAuthenticated)
    {
      userName = user.Identity.Name; // Fetching the signed-in user's name
    }
  }
}
```

4. Update Cosmos DB Integration

When uploading the file, include the user's name from Azure AD:

```
public async Task UploadFileAsync(BlobContainerClient blobContainerClient, string filePath, long fileSize, string
userName)
{
  // File upload logic

  // After upload, save file metadata to Cosmos DB
  var fileData = new
  {
    FileName = filePath,
    FileSize = fileSize,
    UploadTime = DateTime.UtcNow,
    UserName = userName // Save the username from Azure AD
  };

  await cosmosContainer.CreateItemAsync(fileData);
}
```

5. Run the Application

When you run the app, users will be prompted to log in via Microsoft (Azure AD). The app will store the username, file name, file size, and upload time in Cosmos DB after the file upload is complete. You can view telemetry from both the web app and Azure Function in Application Insights.

Deployment

To publish both the Radzen Blazor web app and the Azure Function app to Azure using Visual Studio, follow the steps below:

1. Publish Radzen Blazor Web App to Azure

Step 1: Create an Azure Web App in Azure Portal

1. Go to the Azure Portal.
2. Navigate to **App Services** and click **Create**.
3. Choose **Web App** as the resource.
4. Fill in the necessary details:
    - **Subscription**: Select your Azure subscription.
    - **Resource Group**: Create a new one or select an existing one.
    - **Name**: Provide a unique name for your app (e.g., your-webapp-name).
    - **Publish**: Choose **Code**.
    - **Runtime Stack**: Select .NET Core 7 or the appropriate version based on your app.
    - **Region**: Select your preferred region.
5. Click **Review + Create** and then **Create**.

Step 2: Publish the Web App from Visual Studio

1. Open your Radzen Blazor web app project in Visual Studio.
2. Right-click on the project in **Solution Explorer** and choose **Publish**.
3. In the Publish dialog:
    - Choose **Azure** as the publish target.
    - Select **Azure App Service (Windows)** and click **Next**.
    - Click **Create a new Azure App Service** if you haven't created it yet or select the one you created earlier.
4. Click **Finish** and then click **Publish**.

Visual Studio will build your app and deploy it to Azure App Service. You can monitor the deployment in the **Output** window.

2. Publish Azure Function App to Azure

Step 1: Create an Azure Function App in Azure Portal

1. In the Azure Portal, go to **Create a resource**.
2. Search for **Function App** and click **Create**.

3. Fill in the necessary details:
    - **Subscription**: Select your subscription.
    - **Resource Group**: Select the same resource group as the web app or create a new one.
    - **Function App Name**: Provide a unique name.
    - **Runtime Stack**: Select .NET Core 7 or the appropriate version.
    - **Region**: Select your preferred region.
4. Click **Review + Create** and then **Create**.

Step 2: Publish the Function App from Visual Studio

1. Open your Azure Function app project in Visual Studio.
2. Right-click on the project and select **Publish**.
3. Choose **Azure** as the publish target.
4. Select **Azure Function App** and then **Next**.
5. Choose the **Function App** you created earlier or create a new one.
6. Click **Publish**.

Visual Studio will deploy the Function App to Azure. You can view the status in the **Output** window.

3. Enable Application Insights in Azure

Step 1: Set Up Application Insights for the Web App

1. In the Azure Portal, go to your Web App.
2. Under **Settings**, click on **Application Insights**.
3. If not already enabled, click **Turn on Application Insights**.
4. Follow the prompts to set up a new Application Insights resource or use an existing one.

Step 2: Set Up Application Insights for the Function App

1. In the Azure Portal, go to your Function App.
2. Under **Settings**, click on **Application Insights**.
3. If not already enabled, click **Turn on Application Insights**.
4. Set up the Application Insights resource following the same steps as for the Web App.

4. Configure Managed Identity for Azure Key Vault (Optional)

1. In the Azure Portal, go to your Web App and Function App.
2. Under **Settings**, select **Identity**.

3. Turn on **System-assigned managed identity** for both apps.
4. Go to **Azure Key Vault** and navigate to **Access policies**.
5. Add a new access policy for the managed identities of both the Web App and Function App, granting the necessary permissions (e.g., **Get** and **List** for secrets).
6. Save the changes.

5. Test and Monitor in Application Insights

Once both applications are deployed, you can monitor them via **Application Insights** by:

1. Navigating to the **Application Insights** resource in the Azure Portal.
2. Use the **Logs (Analytics)** to query telemetry data from your applications.